Week 7 Q1:

```
import pandas as pd
data={
    'Name': ['Aman', 'Riya', 'Sohan', 'Priya', 'Karan','Anushka','Raju','Bheem','Chutki'],
    'Age': [22, 21, 23, 20, 24,15,25,18,19],
    'City': ['Delhi', 'Mumbai', 'Pune', 'Chennai', 'Kolkata','Aligarh','Dholapur','Dankor','Gurgaon']
}
df=pd.DataFrame(data)
print(df,"\n")
print("TAIL:\n",df.tail())
print("\nHEAD:\n",df.head())
print(df.info())
```

```
      Name  Age      City
0     Aman   22     Delhi
1     Riya   21    Mumbai
2    Sohan   23      Pune
3    Priya   20   Chennai
4    Karan   24   Kolkata
5  Anushka   15   Aligarh
6     Raju   25  Dholapur
7    Bheem   18    Dankor
8   Chutki   19   Gurgaon

TAIL:
       Name  Age      City
4    Karan   24   Kolkata
5  Anushka   15   Aligarh
6     Raju   25  Dholapur
7    Bheem   18    Dankor
8   Chutki   19   Gurgaon

HEAD:
     Name  Age     City
0   Aman   22    Delhi
1   Riya   21   Mumbai
2  Sohan   23     Pune
3  Priya   20  Chennai
4  Karan   24  Kolkata
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    9 non-null      object
 1   Age     9 non-null      int64
 2   City    9 non-null      object
dtypes: int64(1), object(2)
memory usage: 348.0+ bytes
None
```

Q2:

```
import pandas as pd
import numpy as np

# Create sample DataFrame
data = {
    'A': [10, 20, 30, 40, 50, 60,70,80,90],
    'B': [5, 15, 25, 35, 45, 55,65,75,85],
    'C': [2, 4, 6, 8, 10, 12,9,5,7],
    'D': [1, 3, 5, 7, 9, 11,13,12,15]
}

df = pd.DataFrame(data)
print(df,"\n")

avg2=df.iloc[:,1].mean()
print("Avg of II column: ",avg2)

avg3_4=df.iloc[:5,2:4].mean()
print("Avg of I 5 rows and III and IV col:",avg3_4)

row_sum=df.sum(axis=1)
print("Row-wise sum: ",row_sum)
```

```
max_avg=(df.mean(axis=1)).max()
print("Max Row avg: ",max_avg)
```

```
    A   B   C   D
0  10   5   2   1
1  20  15   4   3
2  30  25   6   5
3  40  35   8   7
4  50  45  10   9
5  60  55  12  11
6  70  65   9  13
7  80  75   5  12
8  90  85   7  15

Avg of II column:  45.0
Avg of I 5 rows and III and IV col: C    6.0
D    5.0
dtype: float64
Row-wise sum:  0     18
1     42
2     66
3     90
4    114
5    138
6    157
7    172
8    197
dtype: int64
Max Row avg:  49.25
```

WEEK 8 Q1:

```
import pandas as pd

data = {
    'col1': ['Aman', 'Riya', 'Sohan'],
    'col2': ['22', '21', '23'],   # stored as string
    'col3': ['85.5', '90.0', '78.0']
}

df = pd.DataFrame(data)
print("Original DataFrame:")
print(df)
print(df.dtypes)

#Change Column name
df.columns=['Name','Age','Marks']
print("\n",df)

#Change DATATYPES
df['Age']=df['Age'].astype(int)
df['Marks']=df['Marks'].astype(float)
print(df.dtypes)
```

```
Original DataFrame:
    col1 col2  col3
0   Aman   22  85.5
1   Riya   21  90.0
2  Sohan   23  78.0
col1    object
col2    object
col3    object
dtype: object

     Name Age Marks
0   Aman  22  85.5
1   Riya  21  90.0
2  Sohan  23  78.0
Name      object
Age        int64
Marks    float64
dtype: object
```

Q2:

```
import pandas as pd
import numpy as np

```

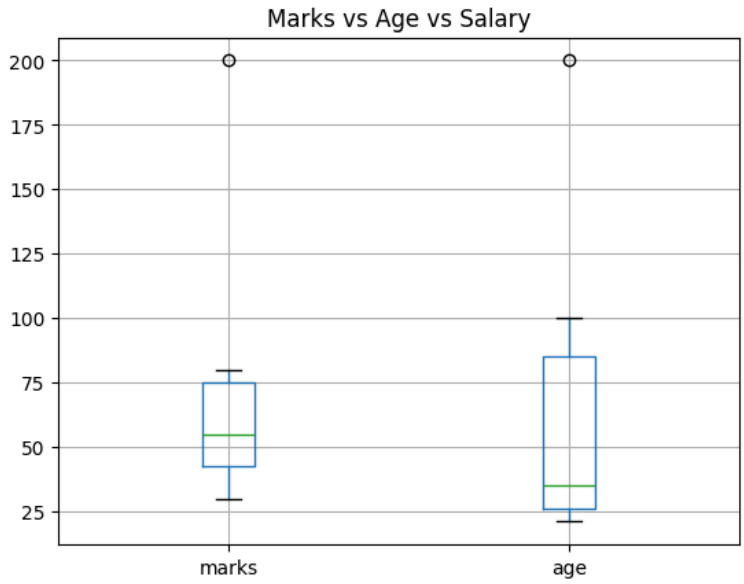
```
data = {
    'Name': ['Aman', 'Riya', 'Sohan', 'Priya', 'Karan'],
    'Marks': [85, 42, 78, None, 30],
    'Age': [22, 21, 23, 20, None]
}

df = pd.DataFrame(data)
print(df,"\n")

print("After removing null rows:\n",df.dropna())

df_new=df[df['Marks']>50]
print("\nRows with marks>50 :\n",df_new)
```

```
    Name  Marks   Age
0   Aman   85.0  22.0
1   Riya   42.0  21.0
2  Sohan   78.0  23.0
3  Priya    NaN  20.0
4  Karan   30.0   NaN

After removing null rows:
     Name  Marks   Age
0   Aman   85.0  22.0
1   Riya   42.0  21.0
2  Sohan   78.0  23.0

Rows with marks>50 :
     Name  Marks   Age
0   Aman   85.0  22.0
2  Sohan   78.0  23.0
```

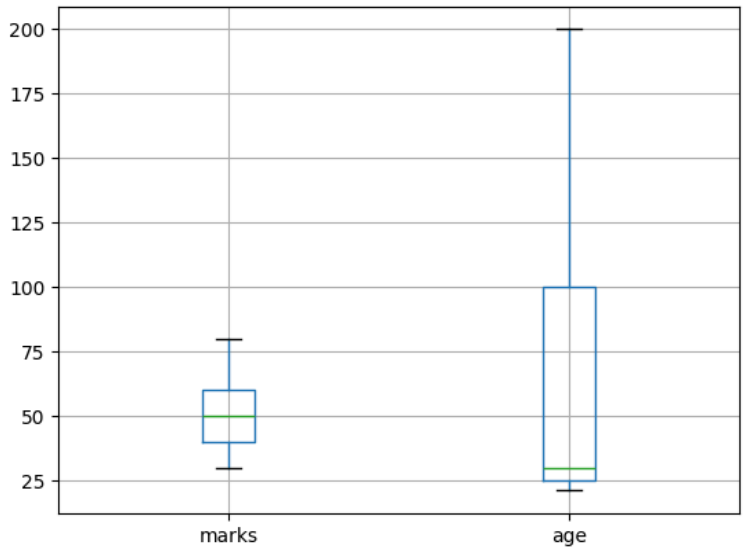
WEEK 9 Q1:

```
import pandas as pd
import matplotlib.pyplot as plt
data={
    'marks':[50,30,60,200,40,80],
    'age':[21,25,30,40,100,200]
}
df=pd.DataFrame(data)
print(df)

df.boxplot()
plt.title("Marks vs Age vs Salary")
plt.show()

df_new=df.copy()
for col in df_new.columns:
  Q1=df_new[col].quantile(0.25)
  Q3=df_new[col].quantile(0.75)
  iqr=Q3-Q1
  lb=Q1-1.5*iqr
  ub=Q3+1.5*iqr

  out=df_new[(df_new[col]<lb) | (df_new[col]>ub)]
  print(f"outliers in {col} are: \n",out)

  #remove outliers
  df_new=df_new[(df_new[col]>=lb) & (df_new[col]<=ub)]

print("cleaned data:\n")
print(df_new)
print("new boxplot:\n")
df_new.boxplot()
plt.show()
```

```
   marks  age
0     50   21
1     30   25
2     60   30
3    200   40
4     40  100
5     80  200
```




```
outliers in marks are:
    marks  age
3    200   40
outliers in age are:
 Empty DataFrame
Columns: [marks, age]
Index: []
cleaned data:

   marks  age
0     50   21
1     30   25
2     60   30
4     40  100
5     80  200
new boxplot:
```



Q2:

```
import pandas as pd

data = {
    'Name': ['Aman', 'Riya', 'Sohan', 'Priya', 'Karan', 'Neha', 'Rahul', 'Simran'],
    'Age': [22, 21, 23, 20, 24, 22, 23, 21],
    'Marks': [85, 90, 78, 92, 88, None, 60, 30],   # 30 = low value
```

```
    'Salary': [30000, 35000, 28000, 40000, 38000, 42000, None, 100000]  # 100000 = outlier
}

df = pd.DataFrame(data)

print("Dataset:\n")
print(df,"\n")

print("HEAD:\n",df.head())
print("\nTAIL:\n",df.tail())
print("\nSHAPE:\n",df.shape)
print("\nCOLUMNS:\n",df.columns)
print("\nMissing values:\n",df.isnull().sum())
print("\nSummary:\n",df.describe())
print("\nCorrelation:\n",df.corr(numeric_only=True))
```

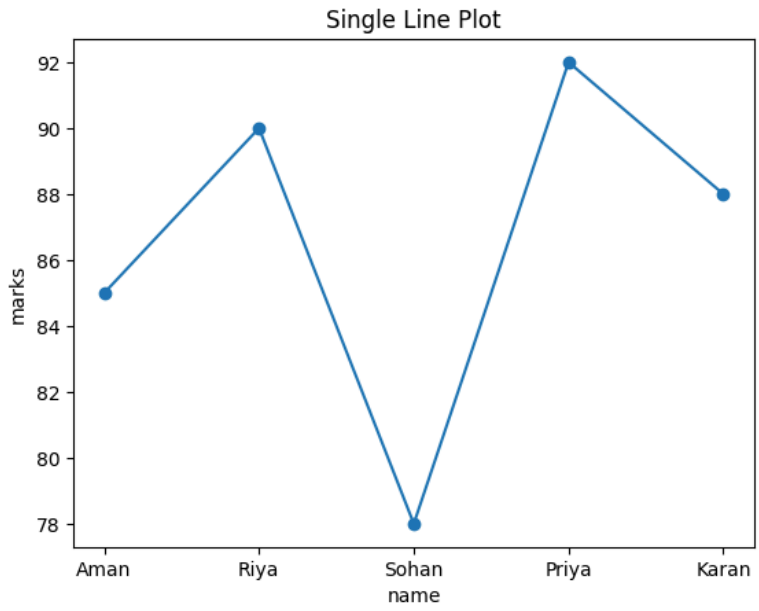
```
Dataset:

     Name  Age  Marks    Salary
0    Aman   22   85.0   30000.0
1    Riya   21   90.0   35000.0
2   Sohan   23   78.0   28000.0
3   Priya   20   92.0   40000.0
4   Karan   24   88.0   38000.0
5    Neha   22    NaN   42000.0
6   Rahul   23   60.0       NaN
7  Simran   21   30.0  100000.0

HEAD:
     Name  Age  Marks   Salary
0   Aman   22   85.0  30000.0
1   Riya   21   90.0  35000.0
2  Sohan   23   78.0  28000.0
3  Priya   20   92.0  40000.0
4  Karan   24   88.0  38000.0

TAIL:
      Name  Age  Marks    Salary
3   Priya   20   92.0   40000.0
4   Karan   24   88.0   38000.0
5    Neha   22    NaN   42000.0
6   Rahul   23   60.0       NaN
7  Simran   21   30.0  100000.0

SHAPE:
 (8, 4)

COLUMNS:
 Index(['Name', 'Age', 'Marks', 'Salary'], dtype='object')

Missing values:
 Name      0
Age       0
Marks     1
Salary    1
dtype: int64

Summary:
              Age      Marks         Salary
count   8.000000   7.000000       7.000000
mean   22.000000  74.714286   44714.285714
std     1.309307  22.514546   24904.579804
min    20.000000  30.000000   28000.000000
25%    21.000000  69.000000   32500.000000
50%    22.000000  85.000000   38000.000000
75%    23.000000  89.000000   41000.000000
max    24.000000  92.000000  100000.000000

Correlation:
              Age     Marks    Salary
Age     1.000000  0.052344 -0.329767
Marks   0.052344  1.000000 -0.933598
Salary -0.329767 -0.933598  1.000000
```

WEEK 10 Q1:

```
import pandas as pd
import matplotlib.pyplot as plt

# Create sample dataset
data = {
    'Name': ['Aman', 'Riya', 'Sohan', 'Priya', 'Karan'],
```

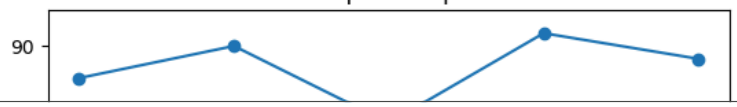
```
    'Marks': [85, 90, 78, 92, 88],
    'Age': [22, 21, 23, 20, 24]
}

df = pd.DataFrame(data)
print(df)

#single line plot
plt.plot(df["Name"],df['Marks'],marker='o')
plt.title("Single Line Plot")
plt.xlabel("name")
plt.ylabel("marks")
plt.show()
print("\n\n")

#multiple Line Plot
plt.plot(df['Name'],df['Marks'],marker="o",label="marks")
plt.plot(df['Name'],df['Age'],marker='o',label='age')
plt.title("Multiple Line plot")
plt.xlabel("Name")
plt.ylabel("Values")
plt.legend()
plt.show()

print("\n\n")

#single bar plot
df.set_index('Name')[['Marks']].plot(kind='bar')
plt.title("Single Bar Plot")
plt.ylabel("Values")
plt.show()

print("\n\n")
#Multiple Bar plot
df.set_index("Name")[["Marks","Age"]].plot(kind='bar')
plt.title("Multiple Bar Plot")
plt.ylabel("values")
plt.show()
```

```
   Name  Marks  Age
0  Aman     85   22
1  Riya     90   21
2 Sohan     78   23
3 Priya     92   20
4 Karan     88   24
```

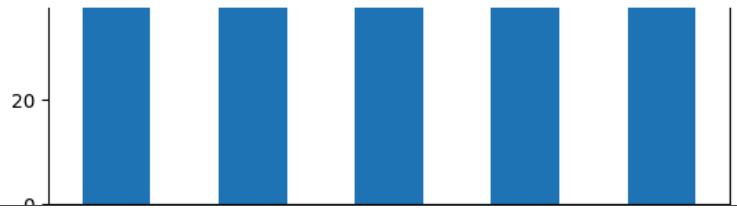


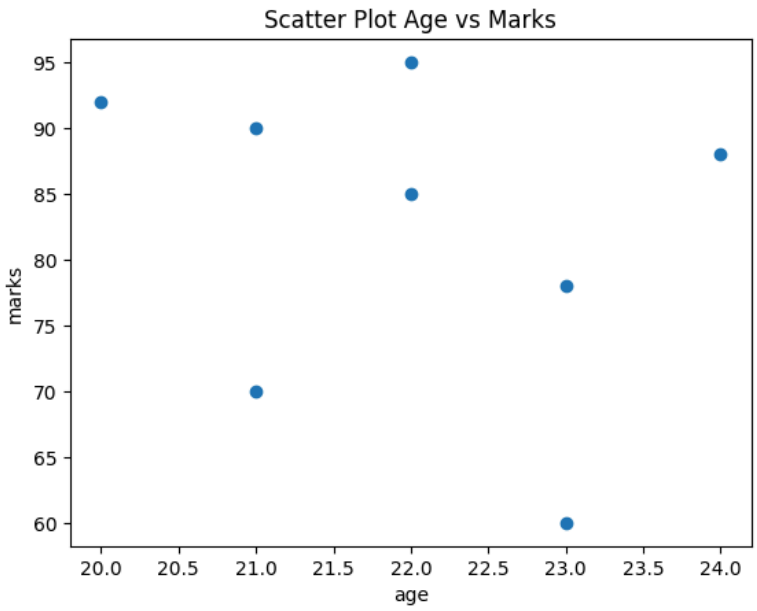
Multiple Line plot

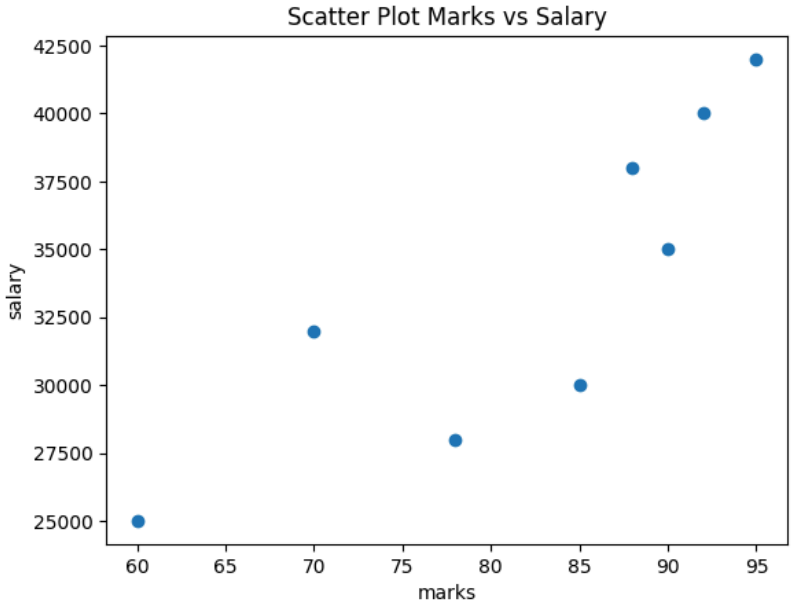
Q2:

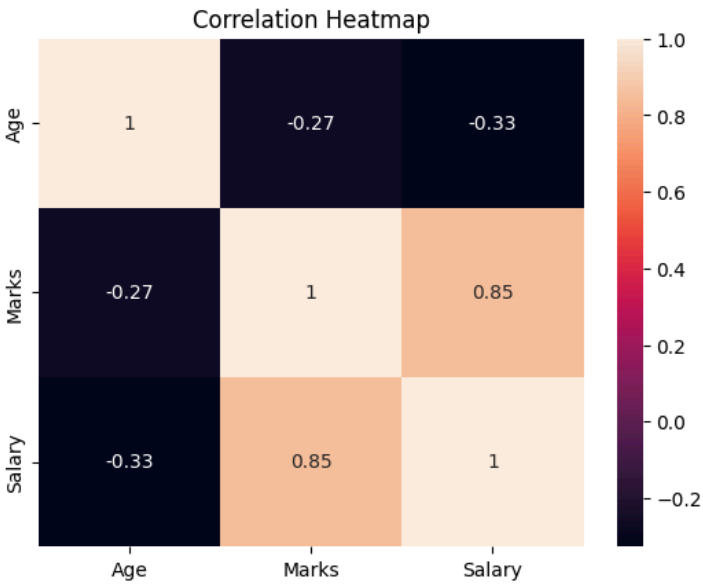



```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

data = {
    'Age': [22, 21, 23, 20, 24, 22, 23, 21],
    'Marks': [85, 90, 78, 92, 88, 95, 60, 70],
    'Salary': [30000, 35000, 28000, 40000, 38000, 42000, 25000, 32000]
}
df=pd.DataFrame(data)

#Scatter plot Age vs Marks
plt.scatter(df['Age'],df['Marks'])
plt.xlabel("age")
plt.ylabel("marks")
plt.title("Scatter Plot Age vs Marks")
plt.show()

print("\n\n")

#Scatter plot marks vs salary
plt.scatter(df['Marks'],df['Salary'])
plt.title("Scatter Plot Marks vs Salary")
plt.xlabel("marks")
plt.ylabel("salary")
plt.show()

print("\n\n")

#Correlation Heatmap
cor=df.corr(numeric_only=True)
sns.heatmap(cor,annot=True)
plt.title("Correlation Heatmap")
plt.show()
```




Scatter Plot Age vs Marks
marks
age
95
90
85
80
75
70
65
60
20.0
20.5
21.0
21.5
22.0
22.5
23.0
23.5
24.0


Scatter Plot Marks vs Salary
salary
marks
42500
40000
37500
35000
32500
30000
27500
25000
60
65
70
75
80
85
90
95


Correlation Heatmap
Age
Marks
Salary
1
-0.27
-0.33
-0.27
1
0.85
-0.33
0.85
1
1.0
0.8
0.6
0.4
0.2
0.0
−0.2

WEEK11 Q1

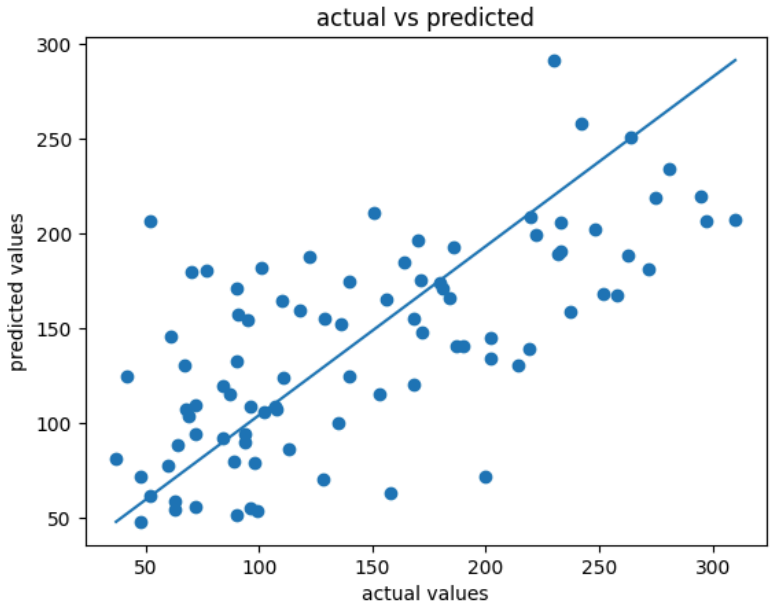
```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris=load_iris()
df=pd.DataFrame(iris.data,columns=iris.feature_names)
df['target']=iris.target

x=df.drop('target',axis=1)
y=df['target']

x_train, x_test, y_train, y_test=train_test_split(x,y,test_size=0.2,random_state=42)
print("x train:",x_train.shape)
print("x test:",x_test.shape)
```

```
x train: (120, 4)
x test: (30, 4)
```

Q2

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

#load
iris=load_diabetes()
df=pd.DataFrame(iris.data, columns=iris.feature_names)
df['target']=iris.target

#split
x=df.drop('target',axis=1)
y=df['target']
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)

#model fit
model=LinearRegression()
model.fit(x_train,y_train)

#prediction
y_pred=model.predict(x_test)

#evaluation
mae=mean_absolute_error(y_test,y_pred)
rmse=np.sqrt(mean_squared_error(y_test,y_pred))
print("mae: ",mae)
print("rmse: ",rmse)

#plot
plt.scatter(y_test,y_pred)
plt.xlabel("actual values")
plt.ylabel("predicted values")
plt.title("actual vs predicted")

#line plot
plt.plot([y_test.min(),y_test.max()],
         [y_pred.min(),y_pred.max()])
plt.show()

```

```
mae:  42.79409467959994
rmse:  53.85344583676593
```



WEEK 12 Q1:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score

# ----------------------------
# 1. Load Dataset
# ----------------------------
data = load_iris()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target

# ----------------------------
# 2. Split Data
# ----------------------------
X = df.drop('target', axis=1)
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ----------------------------
# 3. Train Model
# ----------------------------
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)

# ----------------------------
# 4. Predictions
# ----------------------------
y_pred = model.predict(X_test)

# ----------------------------
# 5. Confusion Matrix
# ----------------------------
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

# ----------------------------
# 6. Metrics
# ----------------------------
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='macro')
```